МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Курсовая работа

по курсу «Программирование графических процессоров»

Обратная трассировка лучей (Ray Tracing) на GPU

Выполнил: А. Р. Немкова
Группа: М8О-408Б-22
Преподаватель: А.Ю. Морозов

Москва,2025

Условие

Цель работы. Использование GPU для создание фотореалистической визуализации. Рендеринг полузеркальных и полупрозрачных правильных геометрических тел. Получение эффекта бесконечности. Создание анимации.
Задание.
Сцена. Прямоугольная текстурированная поверхность (пол), над которой расположены три платоновых тела. Сверху находятся несколько источников света. На каждом ребре многогранника располагается определенное количество точечных источников света. Грани тел обладают зеркальным и прозрачным эффектом. За счет многократного переотражения лучей внутри тела, возникает эффект бесконечности.
Камера. Камера выполняет облет сцены согласно определенным законам. В цилиндрических координатах (r, φ, z), положение и точка направления камеры в момент времени t определяется в соответствии с заданными формулами.
Требуется реализовать алгоритм обратной трассировки лучей с использованием технологии CUDA. Выполнить покадровый рендеринг сцены. Для устранения эффекта «зубчатости», выполнить сглаживание. Полученный набор кадров склеить в анимацию любым доступным программным обеспечением. Подобрать параметры сцены, камеры и освещения таким образом, чтобы получить наиболее красочный результат. Провести сравнение производительности gpu и cpu.
Вариант №6. Тетраэдр, Додекаэдр, Икосаэдр

Программное и аппаратное обеспечение

Графический процессор:
- Модель: NVIDIA GeForce RTX 3050 Laptop GPU
- Compute Capability: 8.6
- Объем графической памяти: 4 294 443 008
- Разделяемая память на блок: 49 152
- Количество регистров на блок: 65 536
- Размер warp: 32
- Максимальное количество потоков на блок: (1024 × 1024 × 64)
- Максимальное количество блоков: (2 147 483 647 × 65 535 × 65 535)
- Объем константной памяти: 65 536
- Количество мультипроцессоров (SM): 16

Центральный процессор:
- Модель: AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz
- Количество ядер: 8
- Количество потоков: 16
- Базовая тактовая частота: 3,2 ГГц
- Максимальная тактовая частота: 4,4 ГГц
- Кеш L1: 64 Кб (на ядро)
- Кеш-память L2: 512 КБ (на ядро)
- Кеш L3: 16 Мб

Оперативная память:
- Тип памяти: DDR4
- Объём памяти: 2 * 8 Гб
- Частота: 3200 Мт/с

Жёсткий диск:

- Тип: SSD
- Модель: SK Hynix HFM001TD3JX013N
- Объём памяти: 1 Тб
- Скорость записи/чтения: 3100/3500 МБ/с

Программное обеспечение:

- OS: Ubuntu 20.04 Linux (WSL2)
- IDE: VS Code
- Compiler: nvcc, gcc

Метод решения

Задача решается методом трассировки лучей. Для каждого пикселя изображения из положения камеры испускается первичный луч в направлении сцены. Луч последовательно проверяется на пересечение со всеми геометрическими объектами сцены. Объектом считается треугольник, так как все поверхности сцены приведены к треугольному виду.
Для луча определяется ближайшая точка пересечения с поверхностью. Если пересечений не найдено, пикселю присваивается фоновый цвет. В случае пересечения вычисляется цвет точки поверхности с учётом локального освещения и отражений.
Основной алгоритм трассировки:
1. Для каждого пикселя вычисляется направление луча на основе параметров камеры.
2. Луч пересекается со всеми треугольниками сцены.
3. Выбирается пересечение с минимальным положительным параметром t.
4. В точке пересечения вычисляется нормаль поверхности.
5. Цвет пикселя определяется моделью освещения.
6. Дополнительно учитывается отражённый луч одного уровня глубины.
Рекурсивная трассировка не используется. Отражение рассчитывается только однократно.
Направление отражённого луча вычисляется по формуле зеркального отражения:

$$r = v - 2 \cdot (v \cdot n) \cdot n$$

где
v — направление падающего луча,
n — нормаль поверхности в точке пересечения,
r — направление отражённого луча.
Отражённый луч используется для дополнительного вычисления освещённости сцены.
Его вклад масштабируется коэффициентом отражения материала.
Используется локальная модель освещения, включающая:
1. Диффузную составляющую
Диффузное освещение рассчитывается по закону Ламберта:

$$I_{diff} = \max(0, n \cdot l)$$

где
n — нормаль поверхности,
l — нормализованный вектор направления на источник света.

Диффузная составляющая умножается на цвет объекта и цвет источника света.
2. Зеркальную составляющую

Зеркальная компонента определяется отражением направления света относительно нормали:

$$I_{spec} = \max(0, r \cdot l_{view})^{\mathrm{k}}$$

где
r — отражённое направление света,
l_{view} — направление на камеру,
k — показатель блеска.
Зеркальная составляющая усиливает блики на отражающих поверхностях.
3. Суммарное освещение
Полный цвет точки поверхности вычисляется как сумма вкладов от всех источников света:

$$C = \Sigma \left(C_{obj} \cdot C_{light} \cdot I_{diff} + C_{light} \cdot I_{spec}\right)$$

Для отражающих поверхностей дополнительно вычисляется цвет отражённого луча.
Итоговый цвет пикселя определяется как линейная комбинация локального освещения и отражения:

$$C_{final} = \left(1 - k_{reflect}\right) \cdot C_{local} + k_{reflect} \cdot C_{reflect}$$

где
$k_{reflect}$ — коэффициент отражения материала.

Для повышения качества изображения применяется метод сверхдискретизации (SSAA). Каждый пиксель исходного изображения разбивается на равномерную сетку под-пикселей. Цвет каждого под-пикселя вычисляется независимо, после чего усредняется:

$$C_{pixel} = \left(\frac{1}{N^2}\right) \cdot \Sigma\, C_{subpixel}$$

где
N — размер сетки суперсэмплинга.

Описание программы

Архитектура программы построена модульно и разделена на логические компоненты, отвечающие за описание сцены, геометрию, камеру, трассировку лучей, освещение и постобработку изображения.
Программа состоит из нескольких файлов исходного кода, собирается с помощью makefile и включает 9 файлов:

- main.cu, makefile
- src/structures.cuh, utils.cuh, camera.cuh, figures.cuh, ray_tracing.cuh, ssaa.cuh

Основные типы данных:

- Vec3, Vec2 — векторные типы для работы с координатами, направлениями и цветами.
- Ray — луч сцены, задаётся началом и направлением.
- Polygon — треугольник сцены с нормалью, цветом, коэффициентами отражения и прозрачности.
- Intersection — информация о пересечении луча с поверхностью: точка, нормаль, цвет, отражение, прозрачность.
- Light — точечный источник света с позицией и цветом.
- Figure — описание фигуры: центр, цвет, радиус, коэффициенты отражения/прозрачности, количество источников света на рёбрах.

- Floor — плоская поверхность сцены с четырьмя углами, цветом, коэффициентом отражения и текстурой.
- Scene — сцена в целом: массив фигур, полигонов, источники света, параметры SSAA и глубины рекурсии, указатели на данные GPU.

Основные функции:
CPU-реализация
- cpuFindIntersection — поиск ближайшего пересечения луча с полигонами сцены.
- cpuCheckShadow — проверка тени для точки сцены.
- cpuComputeShading — вычисление цвета точки сцены по локальной модели освещения (диффуз + зеркальное отражение).
- cpuTraceRaySingleBounce — трассировка луча с учётом одного отражения или прозрачности.
- cpuRenderFrame — рендеринг кадра в CPU-режиме с поддержкой SSAA.

GPU-реализация
- gpuFindIntersection — аналог поиска пересечения, работающий на GPU.
- gpuCheckShadow — проверка тени на GPU.
- gpuComputeShading — вычисление освещения на GPU.
- gpuTraceRaySingleBounce — трассировка луча с одним отражением на GPU.
- gpuTraceKernelSingleBounce — ядро CUDA для параллельной обработки всех пикселей кадра.
- gpuConvertToUChar4 — конвертация цвета из вещественного формата в uchar4 с гамма-коррекцией.
- applySSAA_GPU — ядро для усреднения под-пикселей при включённом SSAA.
- gpuRenderFrame — основная функция рендеринга кадра на GPU.

Вспомогательные функции:
- reflect — вычисление направления отражённого луча.
- rayTriangleIntersect — проверка пересечения луча с треугольником
- loadTexture / freeTexture — работа с текстурами пола.
- createFace, createEdge, createEdgeLightSource — формирование геометрии фигур и источников света.
- buildFigure, buildFloor, buildScene — построение всей сцены и подготовка массивов полигонов и источников света.

Структура программы и разделение по файлам
main.cu - главный файл программы.
В нём выполняется:
- инициализация сцены и камеры;
- выбор режима выполнения (CPU или GPU);
- запуск рендеринга кадров;
- сохранение результата в файл.

structures.cuh - содержит основные структуры данных, используемые во всей программе:
- Vec3, Vec2 — векторные типы для работы с координатами, направлениями и цветами;
- Ray — структура луча, задаваемого начальной точкой и направлением;
- Polygon — треугольник сцены с геометрическими и оптическими свойствами;
- Intersection — структура для хранения информации о пересечении луча с объектом;
- Light — точечный источник света;
- Camera — параметры камеры и генерация первичных лучей;
- Scene, Figure, Floor — описание сцены и её элементов.

utils.cuh - содержит вспомогательные функции и математические операции:

- векторная алгебра;
- отражение вектора относительно нормали;
- проверка пересечения луча с треугольником;

camera.cuh - реализует модель перспективной камеры.
Основная функция камеры — генерация первичных лучей для заданных координат пикселя с учётом:

- положения камеры;
- направления взгляда;
- угла обзора;
- разрешения изображения.

figures.cuh - отвечает за построение геометрии сцены. В файле реализованы:

- генерация вершин и граней правильных многогранников (тетраэдр, додекаэдр, икосаэдр);
- разбиение граней на треугольники;
- построение рёбер в виде тонких параллелепипедов;
- создание точечных источников света вдоль рёбер;
- формирование пола сцены.

Все сложные объекты сцены сводятся к набору треугольников, так как пересечение луча с треугольником является базовой и наиболее универсальной геометрической операцией в трассировке лучей.
ray_tracing.cuh - основной файл, содержащий реализацию алгоритма трассировки лучей.
В файле представлены две независимые версии алгоритма:

- CPU-версия — для последовательного выполнения;
- GPU-версия — для параллельного выполнения с использованием CUDA.

Такое дублирование логики необходимо из-за различий между моделью исполнения CPU и GPU и невозможности прямого использования стандартных контейнеров и рекурсии на GPU.

Описание CUDA-ядер
gpuTraceKernelSingleBounce - основное вычислительное ядро трассировки.
Каждый поток обрабатывает несколько пикселей изображения. Для каждого пикселя:

- Генерируется первичный луч.
- Выполняется поиск пересечения с геометрией сцены.
- Вычисляется освещение в точке пересечения.
- При необходимости рассчитывается отражённый луч одного уровня.
- Записывается итоговый цвет пикселя.

Такой подход обеспечивает равномерную загрузку потоков и эффективно использует параллелизм GPU.
gpuConvertToUChar4 - ядро постобработки изображения.
Выполняет:

- ограничение цветовых значений;
- гамма-коррекцию;
- преобразование цветов из вещественного формата в 8-битный формат uchar4.

applySSAA_GPU - ядро сверхдискретизации изображения.
Осуществляет усреднение высокоразрешённого изображения для получения итогового изображения меньшего размера. Используется при включённом SSAA.

Исследовательская часть и результаты

Были проведены тесты производительности GPU-ядра с различными конфигурациями запуска. Разрешение изображения оставалось постоянным (720×480). Измерялось время построения одного кадра сцены с тремя платоновыми телами (тетраэдр, додекаэдр, икосаэдр) и различным числом источников света на рёбрах. Для сравнения производительности были также выполнены измерения CPU-версии алгоритма.

Таблица 1. Влияние конфигурации CUDA и SSAA на производительность (количество источников света фиксировано – 3)

Конфигурация	SSAA	GPU	CPU
<<1, 32>>	1	530456.3 ms	217 538.8 ms
<<128, 128>>	1	1563.4 ms	
<<256, 256>>	1	1764.8 ms	
<<1, 32>>	2	1753974.7 ms	351782.6 ms
<<128, 128>>	2	5110.4 ms	
<<256, 256>>	2	5319.6 ms	

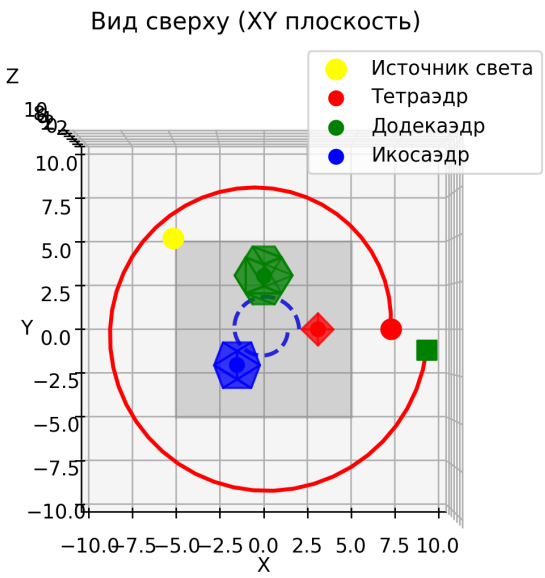
Таблица 2. Влияние числа источников света на время построения кадра (SSAA = 1)

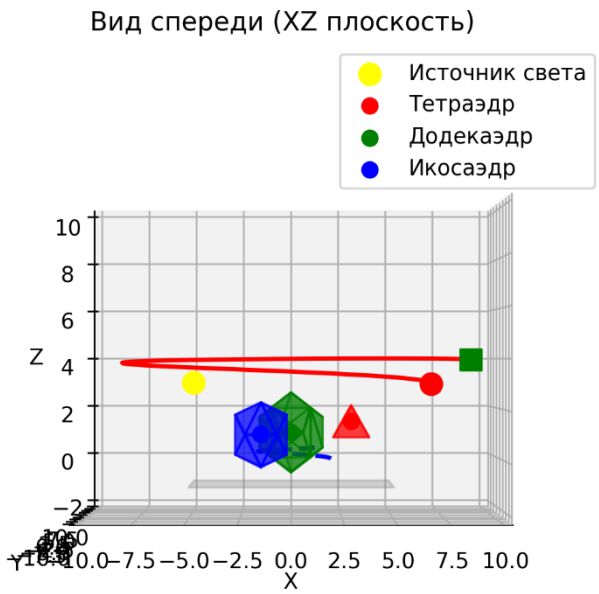
Конфигурация	Число источников света	GPU	CPU
<<128, 128>>	1	365.0 ms	78349.1 ms
<<128, 128>>	2	825.0 ms	161539.2 ms
<<128, 128>>	4	2597.6 ms	400651.5 ms

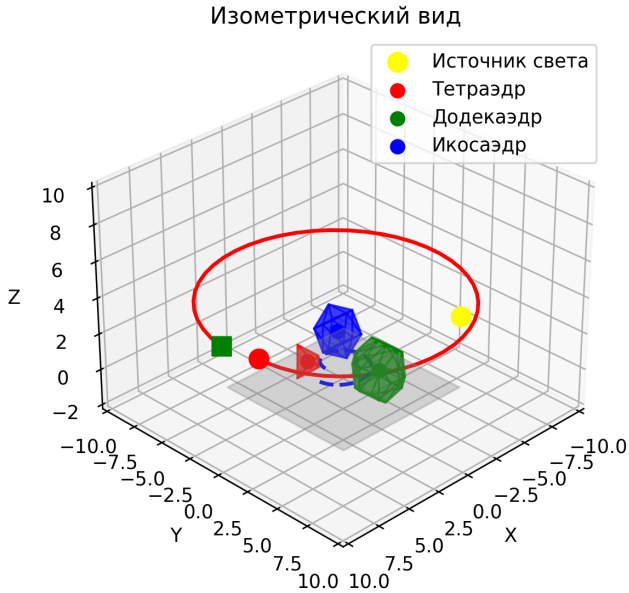
Для получения наиболее выразительного изображения были выбраны следующие параметры:

```
120
output/img_%d.data
720 480 70
7.0 3.0 0.0 2.0 1.0 0.25 0.3 1.0 0.0 0.0
2.0 0.0 0.0 0.8 0.5 0.15 0.2 1.0 3.14 0.0
3 0 1.5 0.9 0.1 0.1 1.5 0.8 0.7 3
0 3 1 0.1 0.9 0.1 1.0 0.8 0.7 3
-1.5 -2 1.0 0.1 0.1 0.9 1.5 0.8 0.7 3
-5 -5 -1 -5 5 -1 5 5 -1 5 -5 -1 texture/floor.data 0.3 0.3 0.3 0.7
1
-5.0 5.0 3.0 0.6 0.6 0.6
1 2
```

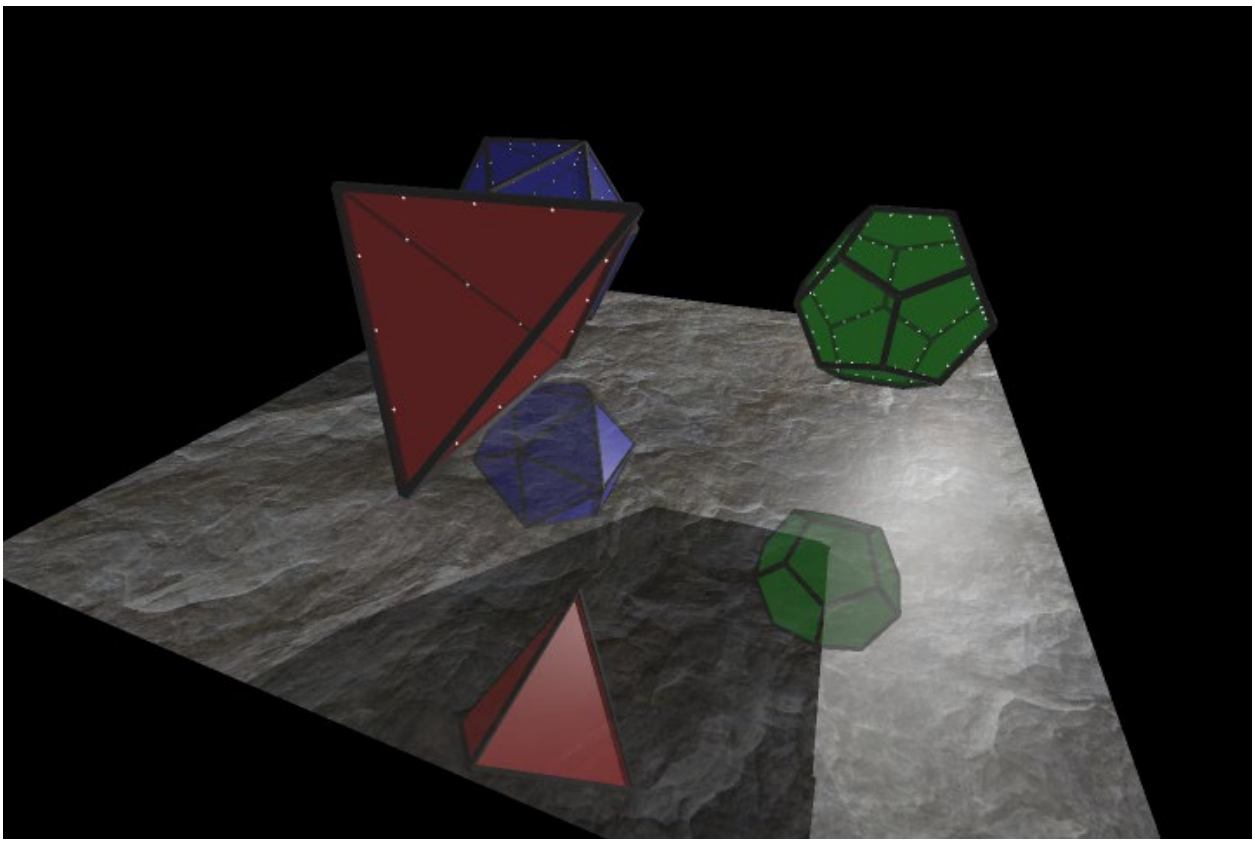
Для анализа сцены построены трёхмерные графики с помощью Matplotlib

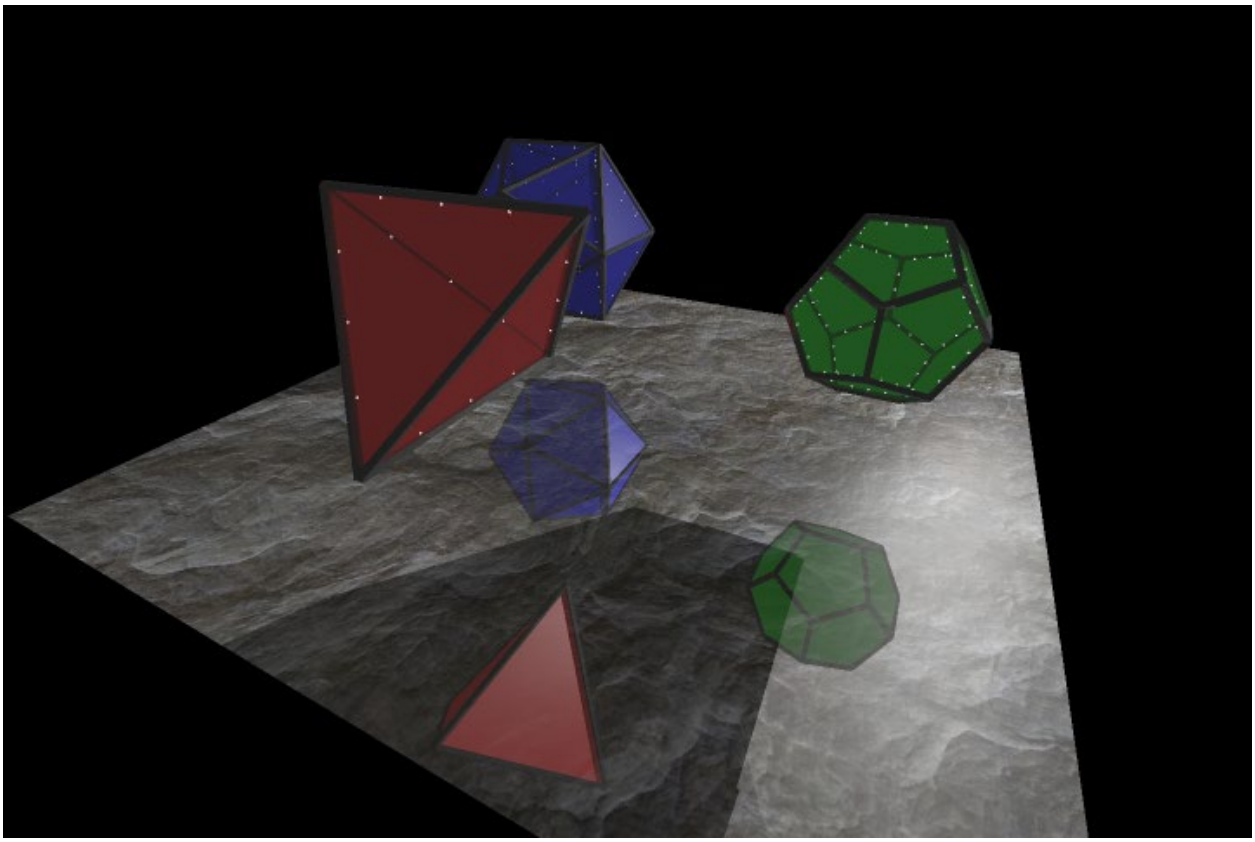


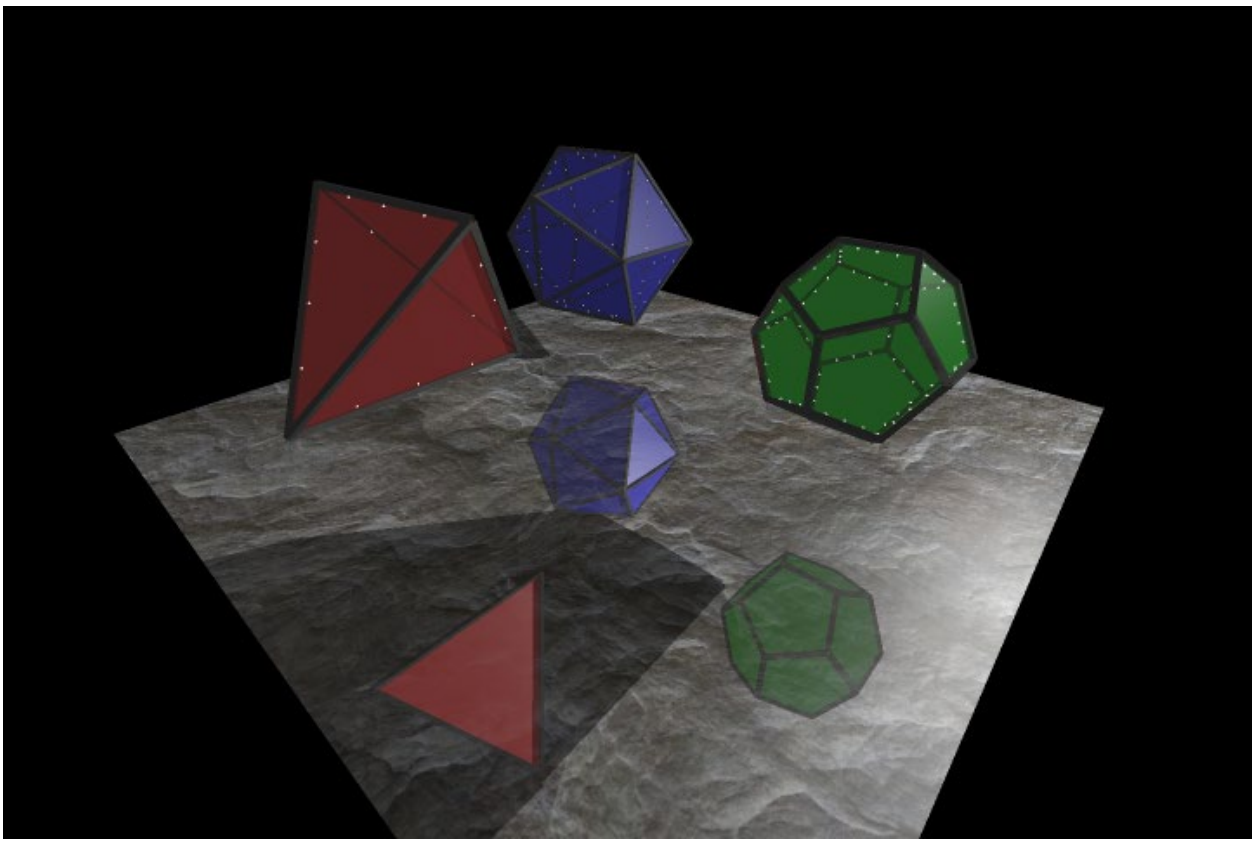


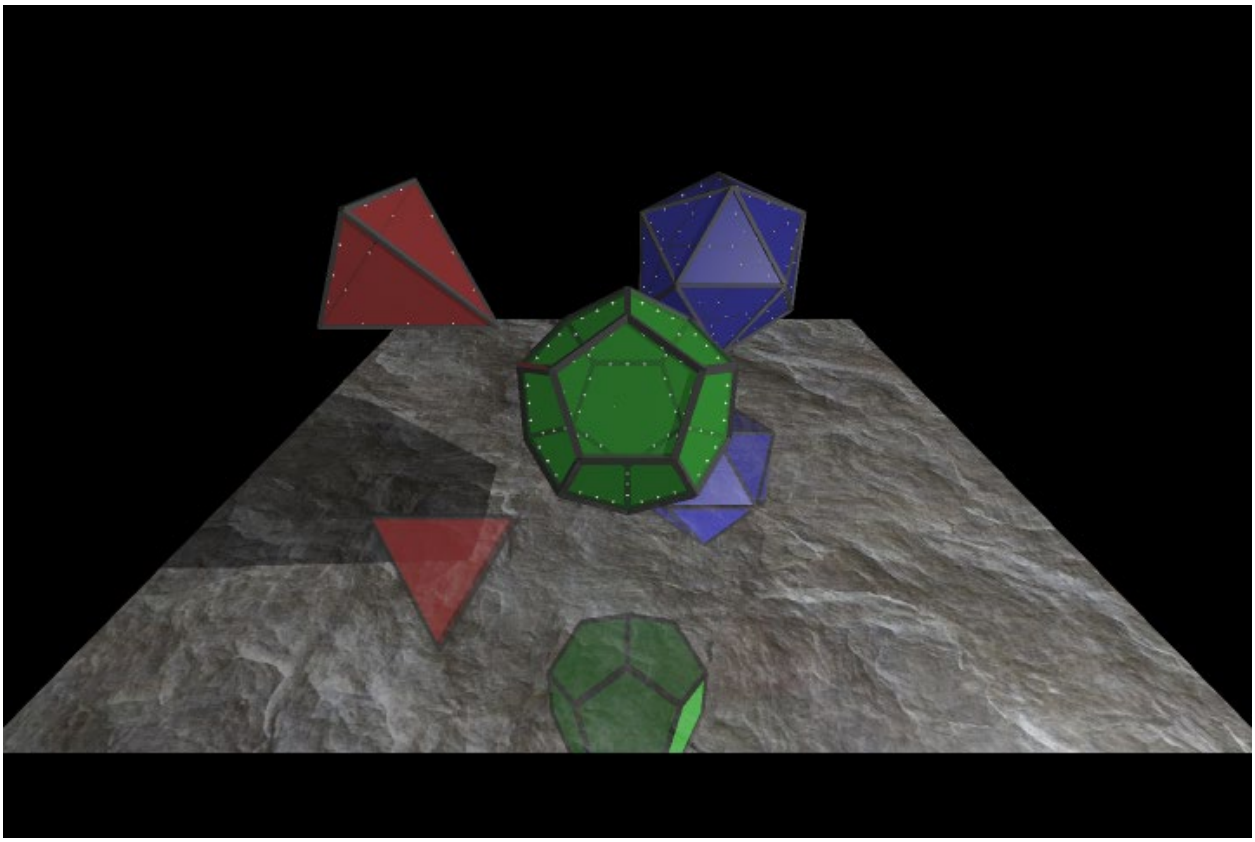


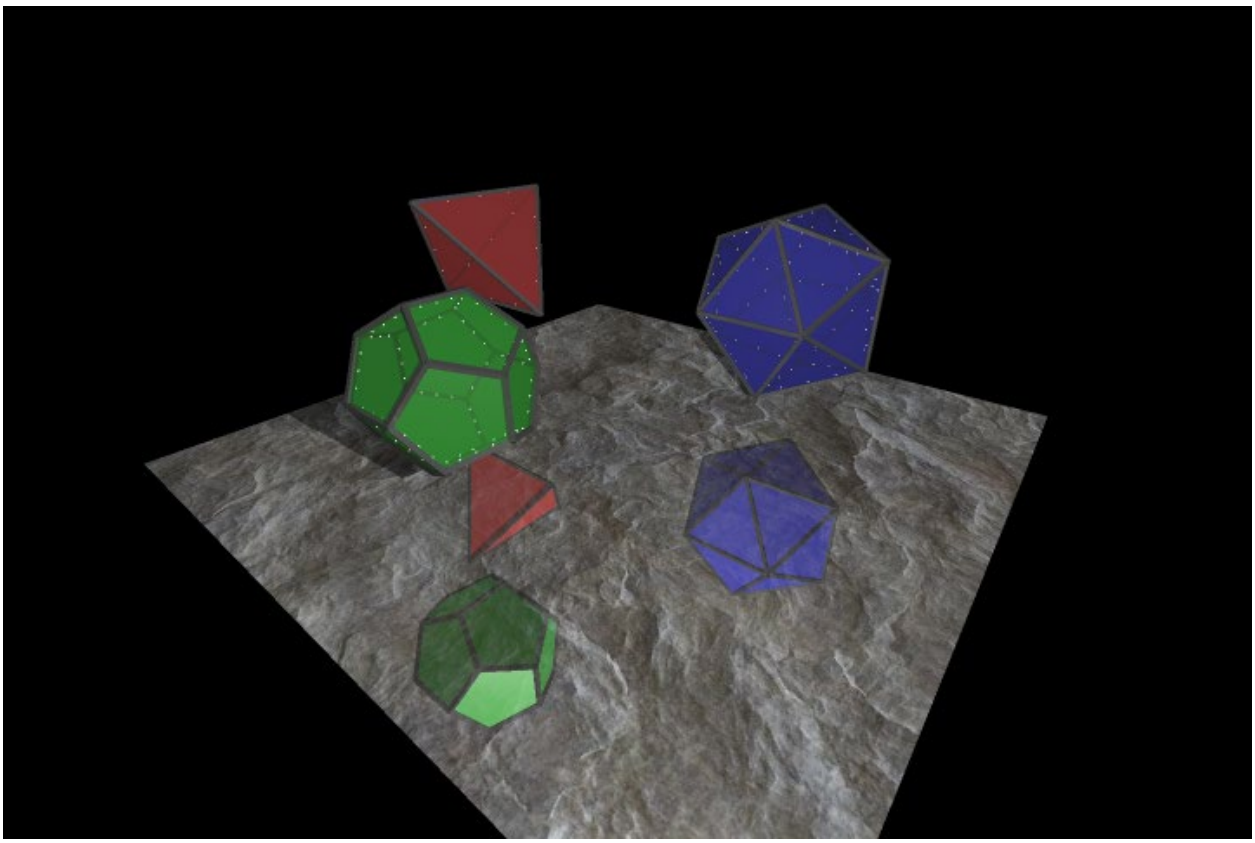
Скриншоты результатов работы

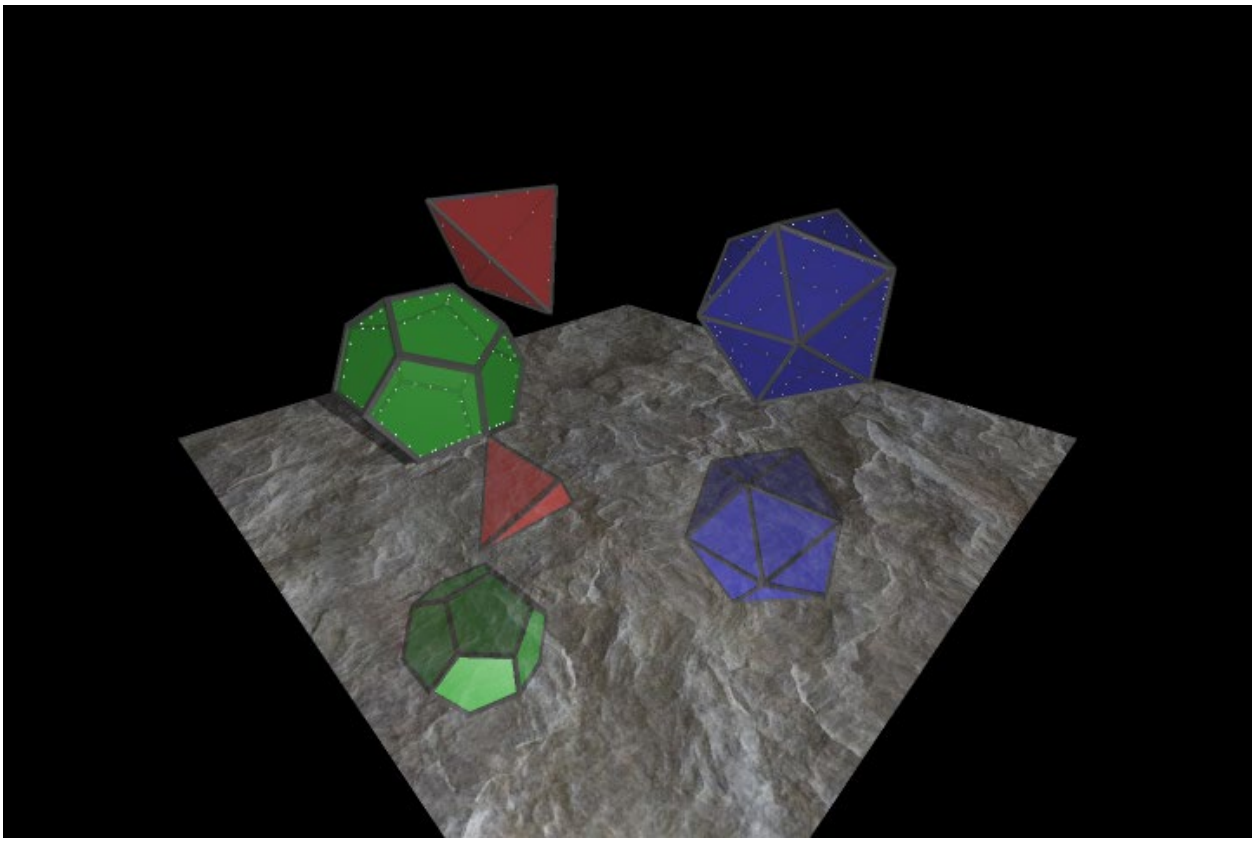


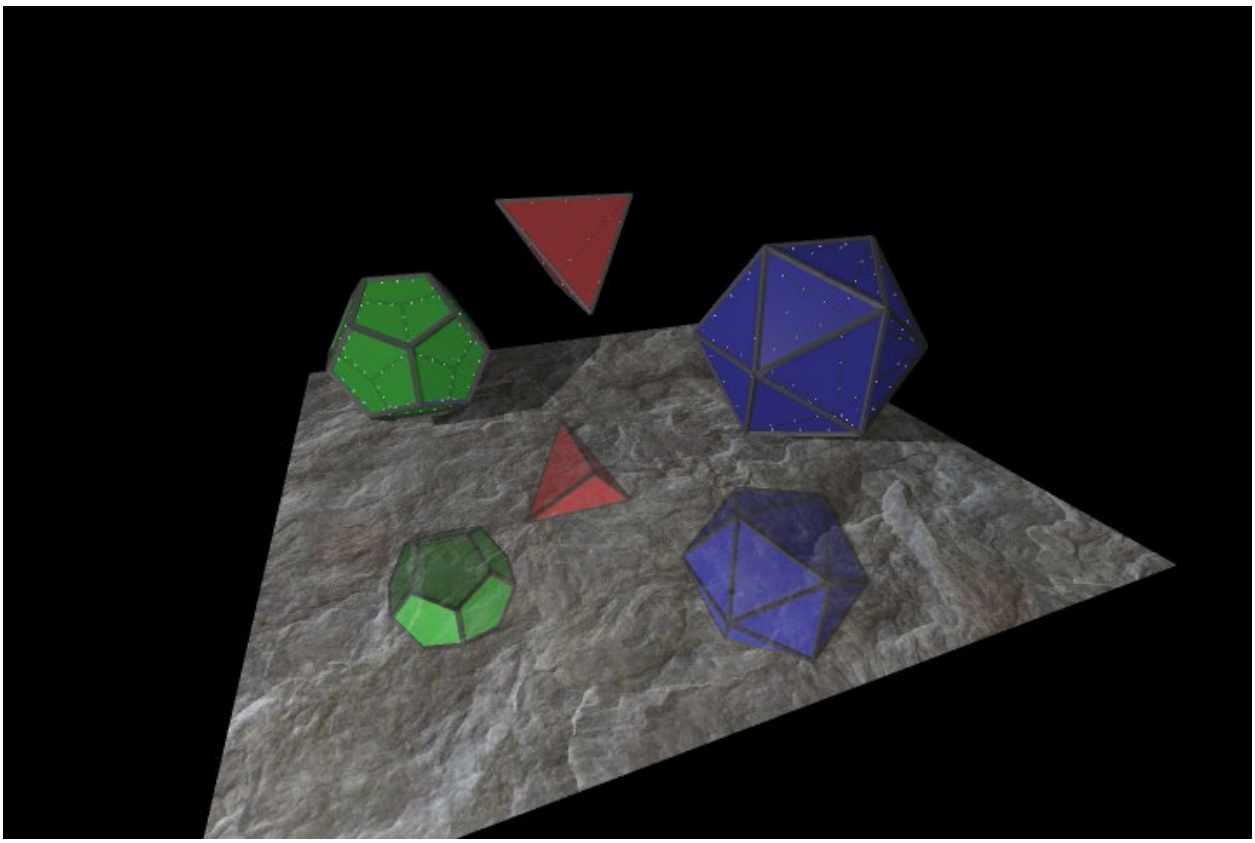


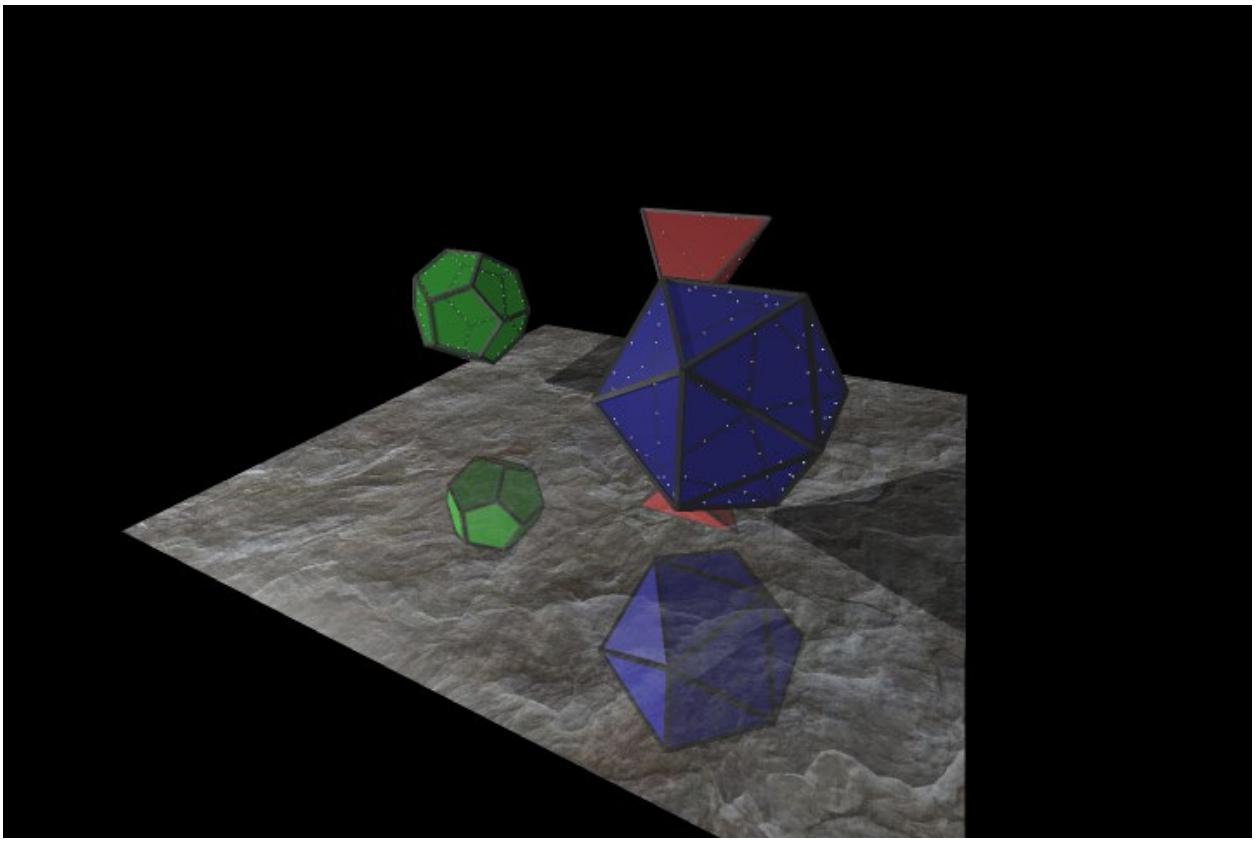


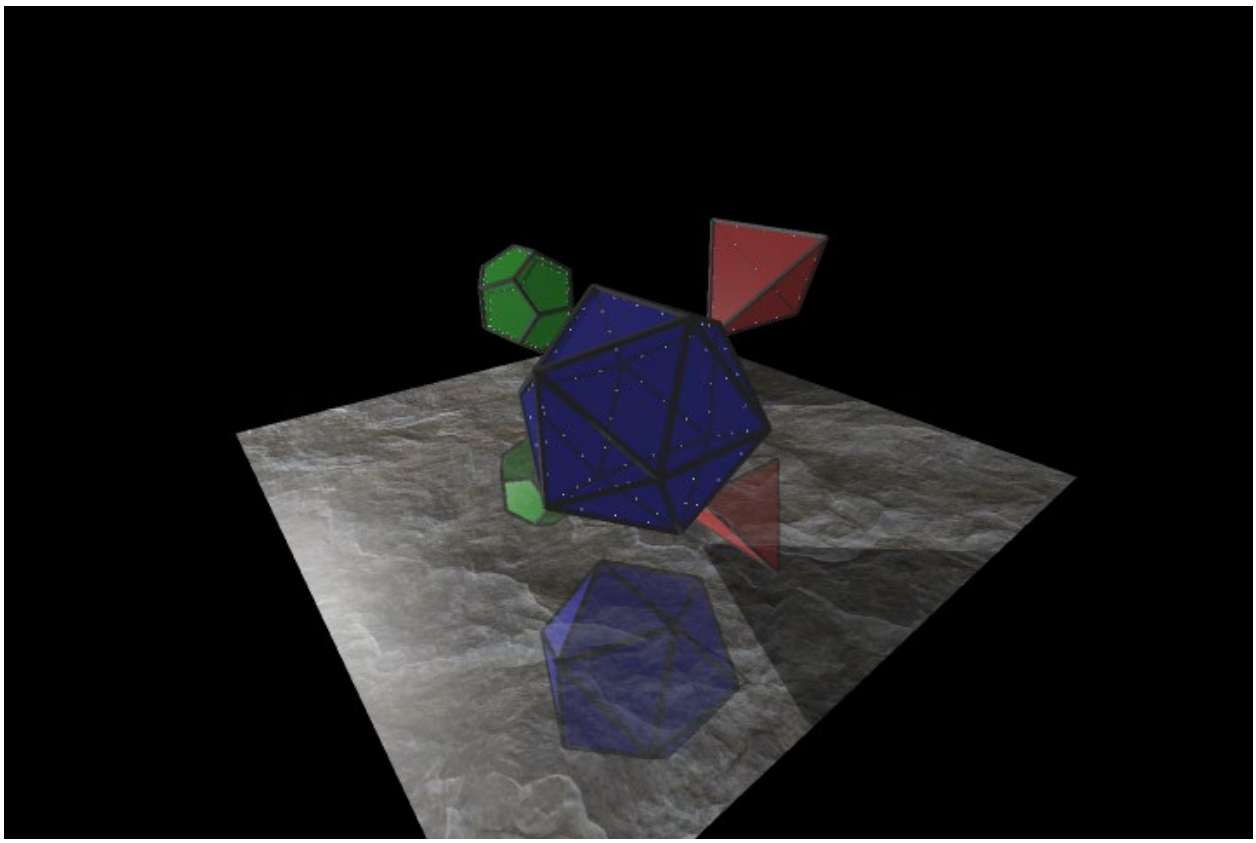


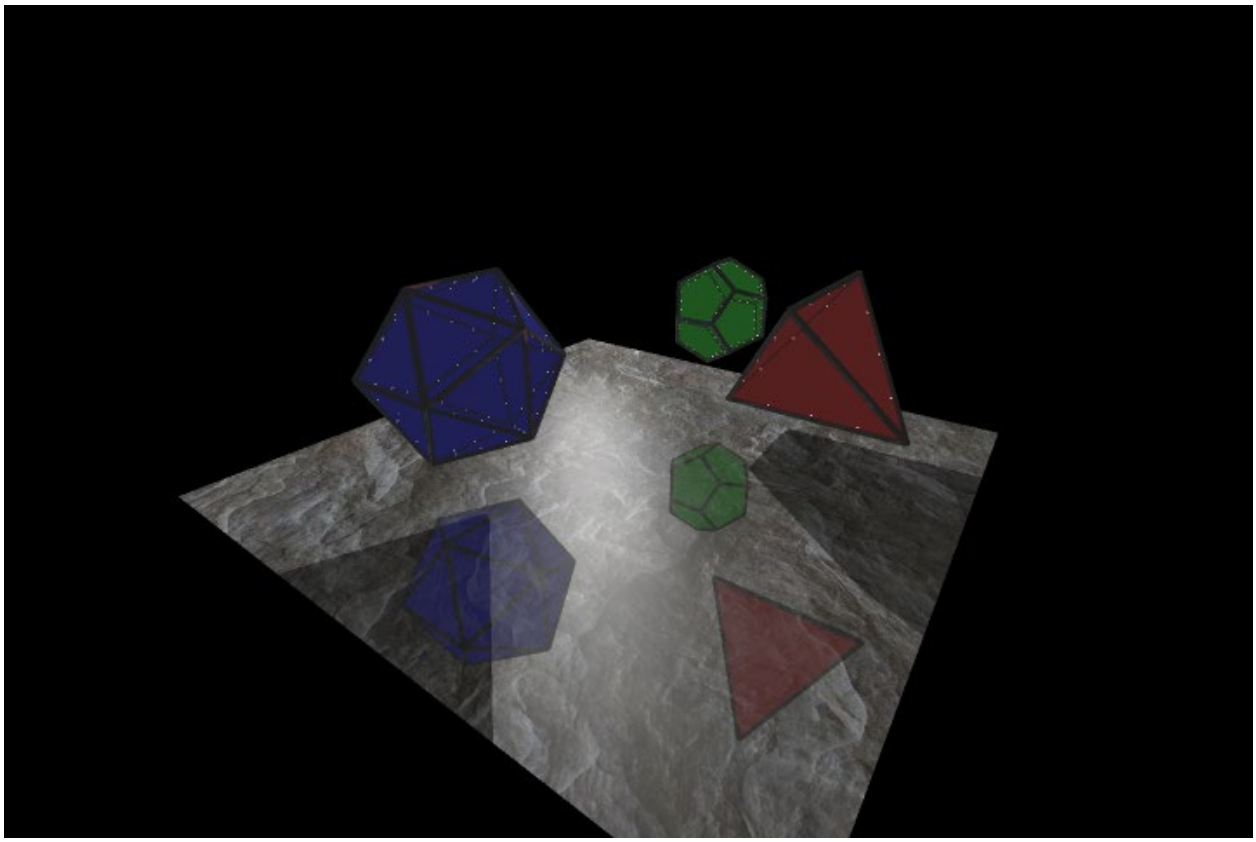












Выводы

В ходе работы была разработана программа для рендеринга сцены с платоновыми телами с использованием метода обратной трассировки лучей и технологии CUDA.
Область применения данного алгоритма включает:

- фотореалистичный рендеринг 3D-сцен для графических симуляторов и анимации;
- визуализацию научных и образовательных моделей многогранников;
- генерацию изображений для дизайна и архитектуры с эффектом зеркальных и прозрачных поверхностей.

Основные трудности связаны с корректной обработкой пересечений лучей с треугольниками, вычислением нормалей и световых эффектов для множества источников, а также с эффективным распределением вычислений на GPU. Использование треугольной аппроксимации поверхностей, одноуровневых отражений и упрощённой модели прозрачности позволило достичь баланса между реалистичностью изображения и производительностью.

Сравнение производительности показало значительное преимущество GPU-версии алгоритма: скорость построения кадра на видеокарте в 20–30 раз выше, чем на CPU, при одинаковых входных данных. Время построения увеличивается пропорционально числу источников света и квадрату числа под-пикселей при включённом SSAA. Конфигурация блоков и потоков CUDA также существенно влияет на производительность, оптимальной оказалась сетка из 128×128 потоков.

Таким образом, реализованный алгоритм демонстрирует высокую эффективность для задач визуализации с отражениями и прозрачностью, обеспечивая красочные и детализированные изображения при разумных затратах вычислительных ресурсов.

Литература

1. http://www.ray-tracing.ru/
2. Platonic solid - Wikipedia
3. https://www.geogebra.org/m/tze9YyJj
4. https://ru.wikipedia.org/wiki/Затенение_по_Фонгу